

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore  

Université Côte d’Azur, Inria, France

Enrico Tassi  

Université Côte d’Azur, Inria, France

Abstract

We mechanize two operational semantics for PROLOG with cut and prove them equivalent in the Rocq prover. The first semantics closely models ELPI’s implementation: it is based on a stack of alternatives and is well suited for studying optimizations employed by PROLOG abstract machines. The second semantics is instead based on And-Or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics to (i) describe introduction and elimination rules for disjunction in the presence of cut, which (as expected) depart from those of classical logic, and (ii) reason about the determinacy analysis introduced in ELPI version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the “UCA DS4H” project, reference ANR-17-EURE-0004.

1 Introduction

ELPI is a dialect of λ PROLOG (see [19, 20, 9, 14]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [15, 3, 4, 26, 10, 11]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [5] and Derive [24, 25, 13], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [12] to help users control backtracking. ROCQ users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [12]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI’s implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [21] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations



© Jane Open Access and Joan R. Public;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:18

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

```

func f int -> int.           % f is a deterministic predicate. The two rules
f 1 2.                       % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int.          % r is a non-deterministic predicate. The heads
r 0 0.                       % overlap: the query for 0 has three applicable
r 0 1.                       % rules.
r 0 2 :- !.
func g int -> int.          % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !.    % second one is cut away, as well as the choice
g D F :- f D E, f E F.       % points left by the call to r.

```

■ **Figure 1** Running example of a ELPI program

used by standard PROLOG abstract machines [27, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1 .

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [12]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceeding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ -calculus style (no parentheses).

The description of a program is given by the record $\mathbb{P}$ and is shared by both semantics. A program consists of a list of rules R together with a signature environment sigT . We delay the discussion of signatures to Section 6 where we describe the static analysis that concerns them.

```

Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.
Record R := mkR { head : Tm; premises : list Atom }.
Record P := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.

```

■ **Figure 2** Definition of term, rule, program and substitution

A rule consists of a head term $\mathbf{Tm}$ and a list of premises. Each premise is an **Atom**, i.e. either the cut operator or a predicate call. Terms $\mathbf{Tm}$ include predicate symbols, data symbols, and variables. The syntax tree allows variables to be applied and predicates to be mixed with data. These higher order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λ Prolog to be based on the same mechanization.

The set of variables $\mathbf{V}$, predicates $\mathbf{P}$ and data symbols $\mathbf{S}$ are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [18]. The empty substitution is written ε .

The backchaining operation applies all rules to a query term; it is central to both semantics.

Definition `backchain` : $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \mathbf{Tm} \rightarrow \Sigma \rightarrow \mathcal{F}_v * \text{list } (\Sigma * \text{list Atom})$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the `backchain` function takes a program, the set of variable names used so far ($\mathcal{F}_v$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization, we abstract over the unifier and its properties. The function `unify` takes two terms t_1 and t_2 , together with a substitution σ , as input, and optionally returns a substitution σ' that extends σ . The expected behavior of `unify` is the standard one (see for example [8, section 2.5]), limited to the first order. Therefore, if t_1 and t_2 unify under σ , then $\sigma' t_1 = \sigma' t_2$, where σt denotes substitution application and $=$ denotes syntactic equality.

2.1 The cut operator

There are quite a few variants of the cut operator. The one we are interested in, i.e. the one used in ELPI, is known as *hard cut* (see, e.g., the documentation of SWI-PROLOG [23]). Whenever the `backchain` function returns a non-empty list of alternatives, the first one is explored immediately, while the others are suspended as choice points. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by `backchain` for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in Figure 1 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$[(\sigma_1, [\mathbf{r} \ \mathbf{A} \ \mathbf{B}, \ \mathbf{f} \ \mathbf{B} \ \mathbf{C}, \ !]), (\sigma_2, [\mathbf{f} \ \mathbf{D} \ \mathbf{E}, \ \mathbf{f} \ \mathbf{E} \ \mathbf{F}])]$

with $\sigma_1 := \{\mathbf{A} \mapsto 0, \mathbf{C} \mapsto \mathbf{R}\}$ and $\sigma_2 := \{\mathbf{D} \mapsto 0, \mathbf{F} \mapsto \mathbf{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_v$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1 , i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\mathbf{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\mathbf{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\mathbf{B} \mapsto 2\}$.

The next step of interpretation continues with `f B C` under σ_3 , that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4 , i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{\mathbf{R} \mapsto 2\}$.

```

Inductive tree :=
  | KO | OK                                     (* failure and success *)
  | Todo of Atom                               (* unexplored branch *)
  | Or  of option tree &  $\Sigma$  & tree          (*  $A \vee B_\sigma$  *)
  | And of tree & list Atom & tree.           (*  $A \wedge_{B_0} B$  *)

```

■ **Figure 3** Definition of And-Or tree

117 We now reach the cut. At this point there are still two unexplored alternatives: one from
 118 the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are
 119 discarded, because they were created when, or after, the first rule for `g` was selected. In
 120 contrast, a cut does not discard choice points created earlier; in this example, there are none.
 121 The final solution is: $\{A \mapsto 0, B \mapsto 1, C \mapsto R, R \mapsto 2\}$.

122 In Sections 3 and 4, we provide fully mechanized operational semantics for PROLOG
 123 with cut. They differ in how they represent the ongoing computation, in particular how
 124 alternatives are stored and how they are pruned by cut.

125 *Notational conventions* In the following section we note $\mathbb{B}$ for the boolean type, and $\top$
 126 and $\perp$ for `true` and `false`. The constructors of the option type are written $\cdot t$ for `Some t`
 127 and $\square$ for `None`. Following the Mathematical Components library, on which we depend,
 128 we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`. List
 129 concatenation is the infix `++`, while list comprehension (i.e. mapping a function on a list) is
 130 written `[seq f x | x <- l]`.

131 3 And-Or Tree semantics

132 An And-Or tree is a stratified, variable-width tree, defined in Rocq as in Section 3. It
 133 consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive
 134 layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer);
 135 for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively,
 136 solving a query requires solving one alternative in the Or-layer, but all subqueries in the
 137 corresponding And-layer.

138 The tree is built incrementally. The `Todo` node represents an unexplored branch (an `Atom`).
 139 When the atom is a `call`, we use the backchaining procedure from Section 2 to compute the
 140 two layers to attach to the tree: alternatives form the Or-layer, and the premises of each
 141 applicable rule form the corresponding And-layer. This expansion step is performed by the
 142 `step` procedure described in Section 3.2.

143 In addition to the `Todo` node we have two distinguished nodes, `KO` and `OK`, which represent
 144 respectively the smallest failed and successful tree.

145 The `Or` node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing
 146 disjunction A . The left subtree A is optional and is absent once that branch has been fully
 147 explored. The right alternative is paired with a substitution σ , which becomes the current
 148 substitution when backtracking to B .

149 The `And` node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first
 150 choice point in A . We call B_0 the *reset point* for B : it represents the continuation for all
 151 alternatives of A except the first. That is, when backtracking occurs within A , the subtree
 152 B is replaced by the atoms in B_0 . An example is shown in Section 3.3.

153 A graphical representation of a tree is shown in Figure 6b. For compactness, we depict

```

Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step :  $\mathbb{P} \rightarrow \mathcal{F} \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F} * \text{tag} * \text{tree})$ 
Definition prune :  $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree}$ 
Inductive runT :  $\mathbb{P} \rightarrow \mathcal{F} \rightarrow \Sigma \rightarrow \text{tree} \rightarrow \text{option } (\Sigma * \text{option tree}) \rightarrow \mathbb{B} \rightarrow \mathcal{F} \rightarrow \text{Prop}$ 

```

■ **Figure 4** Signatures of the tree-based semantics

154 And and Or nodes as n -ary (rather than binary), with children associating to the right. The
 155 KO node acts as the terminating element of an Or list, while OK is the terminating element of
 156 an And list.

157 3.1 The tree-based semantics

158 The tree is constructed incrementally by two recursive functions, **step** and **prune**. Both
 159 traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved
 160 thanks to the **next_tree** (in Figure 5). When this node is OK we say the entire tree is a
 161 *success*, when it is KO we say the tree *failed*, otherwise the tree is *incomplete*. In Figure 6 we
 162 mark with a black arrow the result of **next_tree**.

163 Since all functions must terminate in Rocq, we define their transitive closure of **step**
 164 and **prune** as the inductive relation **runT**. More precisely **runT** iterates calls to **step** on
 165 incomplete trees to expand **Todo** nodes. When a tree fails it calls **prune** to find the next
 166 alternative and continues from there, if one exists.

167 3.2 The step procedure

168 The **step** procedure takes as input a program, a set of variables, a substitution, and a tree,
 169 and returns an updated set of variables, a **tag**, and an updated tree.

170 The set of variables is assumed to contain all variables occurring in the tree. It is passed
 171 to backchain so that rules can be refreshed correctly.

172 The **tag** (see Figure 4) indicates which kind of step was performed. **Failure** and **Success**
 173 are returned for trees that satisfy, respectively, the **failed** and **success** tests. **CutBrothers**
 174 indicates the interpretation of a superficial cut: **step** was called on an And-layer and its

¹ The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2 , and R_3 are reset points and correspond respectively to the lists of atoms $[f \ Y \ Z, !]$, $[!]$, and $[f \ Y \ Z]$.

```

Fixpoint next  $\sigma \ t : \Sigma * \text{tree} :=$ 
  match  $t$  with
  | Todo _ | KO | OK  $\Rightarrow (\sigma, t)$ 
  | Or  $\square \ \sigma' \ B \Rightarrow \text{next } \sigma' \ B$ 
  | Or  $\cdot A \ _ \ _ \Rightarrow \text{next } \sigma \ A$ 
  | And  $A \ _ \ B \Rightarrow$ 
    let  $(\sigma', pA) := \text{next } \sigma \ A$  in
    if  $pA == \text{OK}$  then  $\text{next } \sigma' \ B$ 
    else  $(\sigma', pA)$ 
  end.

Definition next_subst  $\sigma \ t := (\text{next } \sigma \ t).1.$ 
Definition next_tree  $t := (\text{next } \varepsilon \ t).2.$ 

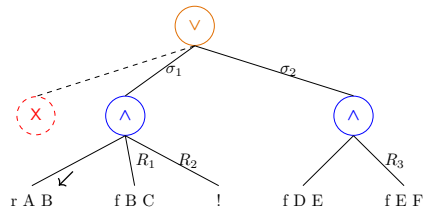
Definition success  $t := \text{next\_tree } t == \text{OK}.$ 
Definition failed  $t := \text{next\_tree } t == \text{KO}.$ 
Definition incomplete  $t :=$ 
  if  $\text{next\_tree } t$  is Todo _ then  $\top$  else  $\perp$ .

```

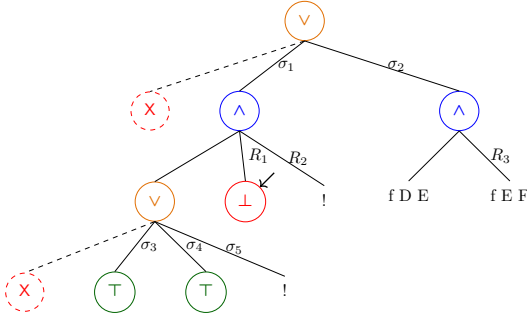
■ **Figure 5** next and derived functions

g 0 R

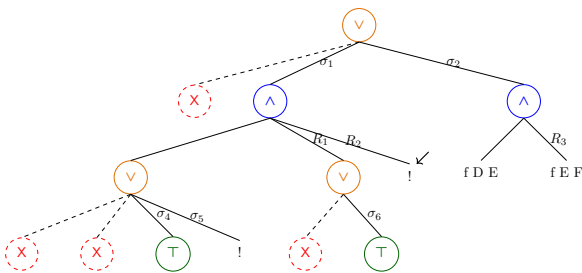
(a) Initial query



(b) Tree after step on g 0 R

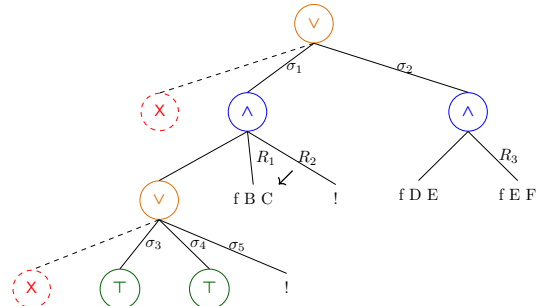


(d.1) step on Figure 6c

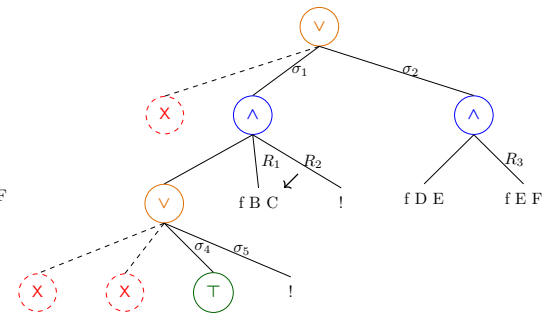


(e) step on Figure 6d.2

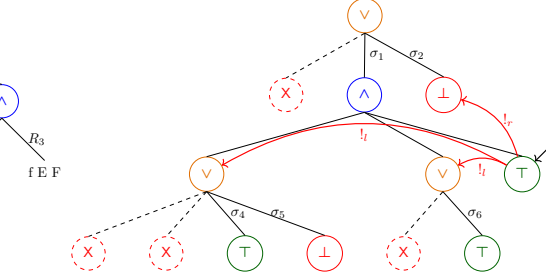
Figure 6 Some tree representations¹



(c) step on Figure 6b



(d.2) prune on Figure 6d.1



(f) step on Figure 6e

175 **next_tree** is the cut atom. Finally, **Expanded** accounts for the interpretation of predicate
 176 calls and non-superficial cuts.

177 The **step** procedure evaluates atoms. In particular, it leaves the tree unchanged when
 178 the tree is *success* or *failed*.

179 ► **Lemma 1** (*succ_step_iff*). $\forall p \ v \ \sigma \ t, \ \text{success } t \leftrightarrow \text{step } p \ v \ \sigma \ t = (v, \text{Success}, t).$

180 ► **Lemma 2** (*fail_step_iff*). $\forall p \ v \ \sigma \ t, \ \text{failed } t \leftrightarrow \text{step } p \ v \ \sigma \ t = (v, \text{Failed}, t).$

181 The **step** procedure expands **Todo** nodes by calling **backchain**, which returns a list l of
 182 alternatives. If l is empty, then the current call atom is replaced by **K0**. Otherwise, it is
 183 replaced by a right-skewed tree of **Or** nodes, where each leaf corresponds to a list of atoms
 184 $e \in l$. If e is empty, the leaf is **OK**; otherwise, it is a right-skewed tree of **And** nodes.

185 Figure 6 depicts the tree corresponding to the running example (Section 2.1) as it
 186 undergoes calls to **step**. We run query $q \ 0 \ R$ against the program P in Figure 1. Let c_0 ,
 187 c_1 , and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2
 188 correspond respectively to the premises of rules r_1 and r_2 in P . Note that c_1 (resp. c_2) is
 189 associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset
 190 points are defines as follows: $R_1 := [\mathbf{f} \ Y \ Z, !]$, $R_2 := [!]$, and $R_3 := \mathbf{f} \ Y \ Z$. Intuitively, they
 191 record the lists of atoms used to generate the corresponding subtree. Other examples of
 192 **step** performing a tree expansion via **backchain** appear in Figures 6c–6e.

193 3.2.1 The interpretation of a cut

194 The step corresponding to a cut performs two actions: (i) the cut node is replaced by **OK**; and
 195 (ii) the subtrees in the scope of **Cut** are pruned. More precisely, (ii) prunes the left siblings
 196 of the **Cut** node (in the same **And**-layer, denoted $!_l$) and prunes the right uncles of **Cut** (in
 197 the one-level-up **Or**-layer, denoted $!_r$).

198 An example of the impact of cut is shown in Figure 6f. The red arrows indicate the
 199 scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied
 200 to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the
 201 cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under
 202 substitution σ_5 (which also succeeds) is replaced by **K0**. This amounts to discarding the third
 203 rule of predicate **r**.

204 3.3 The prune procedure

205 When **backchain** returns an empty list, **step** produces a tree that *failed* and the computation
 206 must backtrack. The **prune** procedure is responsible for producing the new tree from which
 207 the computation can continue.

208 In Figure 6d.1 we encounter a failure because no rule applies to the atom $\mathbf{f} \ B \ C$ under
 209 substitution σ_3 , which maps **B** to 0. The **prune** procedure prunes the path leading to this
 210 failure and resets the current sub-query to its original shape. More precisely, it kills the **OK**
 211 node under σ_3 (since that branch has been fully explored and produced no solution), and
 212 then replaces the **K0** node with the information stored in the reset point R_1 . This restores
 213 the call $\mathbf{f} \ B \ C$, which can then be reconsidered under substitution σ_4 .

214 Furthermore, **prune** serves another important purpose. Its behavior depends on the
 215 boolean flag it receives as input: **prune** $\perp t$ recursively removes all failures (if any) from t ,
 216 whereas **prune** $\top t$ removes the first success in t . For example, the tree in Figure 6f contains
 217 a successful path that maps **R** to 2. Calling **prune** on this tree returns $\square$, since there are no
 218 alternatives in the tree after the success.

$$\begin{array}{c}
\frac{\text{success } t \quad \text{next_subst } \sigma \ t = \sigma' \quad \text{prune } \top \ t = t'}{\text{runT } p \ v \ \sigma \ t \cdot(\sigma', t') \perp v} \text{StopT} \\
\\
\frac{\text{incomplete } t \quad b' = (\text{tg} = \text{CutBrothers}) \vee b \quad \text{step } p \ v \ \sigma \ t = (v', \text{tg}, t') \quad \text{runT } p \ v' \ \sigma \ t' \ r \ b \ v''}{\text{runT } p \ v \ \sigma \ t \ r \ b' \ v''} \text{StepT} \\
\\
\frac{\text{failed } t \quad \text{prune } \perp \ t = \cdot t' \quad \text{runT } p \ v \ \sigma \ t' \ r \ n \ v'}{\text{runT } p \ v \ \sigma \ t \ r \ n \ v'} \text{BackT} \\
\\
\frac{\text{prune } \perp \ t = \square}{\text{runT } p \ v \ \sigma \ t \ \square \perp v} \text{FailT}
\end{array}$$

■ **Figure 7** Rule system for `runT`

Some interesting properties of `prune` are shown below; they illustrate how `prune` complements `step` with respect to failed, successful, and incomplete trees.

► **Lemma 3** (`incomplete_prune_id`). $\forall b \ t, \text{incomplete } t \rightarrow \text{prune } b \ t = \cdot t$.

► **Lemma 4** (`prune_Some`). $\forall b \ t \ t', \text{prune } b \ t = \cdot t' \rightarrow \text{failed } t' = \perp$.

► **Lemma 5** (`prune_None`). $\forall t, \text{prune } \perp \ t = \square \rightarrow \text{failed } t$.

3.4 The `runT` inductive

The inductive relation `runT` relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ , and a tree t . The outputs are an optional result r , a boolean b , and an updated set of variable names.

The main result r is $\square$ when the execution fails; otherwise when $r = \cdot(\sigma', t')$ the execution succeeded and produced the solution σ' ; t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution.

The `runT` predicate has four cases, depicted as inference rules in Figure 7. (*StopT*) If `next_tree` t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. (*StepT*) If `next_tree` t is an atom, then `step` is invoked to evaluate t , and `runT` is called recursively on the updated tree. (*BackT*) If `next_tree` t is a failure, then `prune` is called to clear the failed path; if the resulting cleaned tree exists, `runT` is called recursively on it. Finally, (*FailT*) accounts for trees with no solutions.

The set of rules defining `runT` is deterministic.

► **Lemma 6** (`runT_det`). $\forall p \ v_0 \ \sigma \ t \ r \ r' \ b \ b' \ v \ v', \text{runT } p \ v_0 \ \sigma \ t \ r \ b \ v \rightarrow \text{runT } p \ v_0 \ \sigma \ t \ r' \ b' \ v' \rightarrow r' = r \wedge v' = v \wedge b' = b$.

The proof is by induction on the first `runT` derivation and is immediate, because the rules are selected based on `next_tree` t . In particular, the hypotheses `success` t , `incomplete` t , and `failed` t are mutually exclusive. Moreover, by Lemma 5, the hypothesis of *FailT* implies that t failed; hence *FailT* is exclusive with *StopT* and *StepT*, and it is also exclusive with *BackT* since they impose different requirements on the output of `prune`.

The boolean output of `runT` allows us to state interesting properties about the `Or` node in the presence of cut. Here we report only a selection of those we proved.

► **Lemma 7** (`runSST_or`). $\forall p \ v \ v' \ \sigma \ \sigma' \ A \ A' \ \sigma_1 \ B, \text{runT } p \ v \ \sigma \ A \cdot(\sigma', \cdot A') \top v' \rightarrow$

$\text{runT } p \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma', \cdot (\cdot A' \vee KO_{\sigma_1})) \perp v'.$

247 ▶ **Lemma 8** (`runNT_or`). $\forall p \ v \ v' \ \sigma \ A \ \sigma_1 \ B, \text{runT } p \ v \ \sigma \ A \ \square \top v' \rightarrow$
 $\text{runT } p \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \ \square \perp v'.$

248 ▶ **Lemma 9** (`runNF_or`). $\forall p \ v_0 \ v_1 \ v_2 \ \sigma \ A \ \sigma_1 \ \sigma_2 \ B \ B' \ b,$
 $\text{runT } p \ v_0 \ \sigma \ A \ \square \perp v_1 \rightarrow \text{runT } p \ v_1 \ \sigma_1 \ B \cdot (\sigma_2, B') \ b \ v_2 \rightarrow$
 $\text{let } sR := \text{omap } (\text{fun } x \Rightarrow \square \vee x_{\sigma_1}) \ B' \text{ in}$
 $\text{runT } p \ v_0 \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma_2, sR) \perp v_2.$

249 Lemmas 7–9 correspond to the introduction rules for disjunction. If A executes a cut, one
 250 *cannot* obtain $A \vee B$ from B : the execution of A , whether it ultimately succeeds or fails, makes the
 251 success of B irrelevant (the cut discards it). In the first lemma, B is forced to be $K0$; in the second
 252 one, the failure of A absorbs B . The last lemma is connected to the depth-first exploration of the
 253 tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been
 254 disproved. Depicted as rules:

$$\frac{A \quad (A \text{ cuts})}{A \vee \top} \quad \frac{\neg A \quad (A \text{ cuts})}{\neg(\neg A \vee B)} \quad \frac{\neg A \quad B \quad (A \text{ does not cut})}{A \vee B}$$

255 Lemmas 10 and 11 are elimination rules for disjunction.

256 ▶ **Lemma 10** (`run_orSST`). $\forall p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ A \ A' \ B \ B',$
 $\text{runT } p \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma', \cdot (\cdot A' \vee B'_{\sigma_1})) \perp v' \rightarrow$
 $\exists b, \text{runT } p \ v \ \sigma \ A \cdot (\sigma', \cdot A') \ b \ v' \wedge B' = \text{if } b \text{ then } KO \text{ else } B.$

257 ▶ **Lemma 11** (`run_orSNT`). $\forall p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ A \ B \ B',$
 $\text{runT } p \ v \ \sigma \ (\cdot A \vee B_{\sigma_1}) \cdot (\sigma', \cdot (\square \vee B'_{\sigma_1})) \perp v' \rightarrow$
 $(\exists b, \text{runT } p \ v \ \sigma \ A \cdot (\sigma', \square) \ b \ v' \wedge \text{if } b \text{ then } B' = KO \text{ else } \text{prune } \perp \ B = \cdot B') \vee$
 $(\exists v_2 \ b, \text{runT } p \ v \ \sigma \ A \ \square \perp v_2 \wedge \text{runT } p \ v_2 \ \sigma_1 \ B \cdot (\sigma', \cdot B') \ b \ v').$

258 From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B
 259 independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B'
 260 provides no information in the case where the exploration of A performs a cut. This yields an
 261 elimination rule that is stronger than the usual one (depicted on the left). If A does not cut, the
 262 elimination rule has the usual two premises, together with the additional constraint that whenever
 263 B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

$$\frac{A \vee B \quad \begin{array}{c} [A] \\ C \end{array} \quad (A \text{ cuts})}{C} \quad \frac{A \vee B \quad \begin{array}{c} [A] \\ C \end{array} \quad \begin{array}{c} [\neg A, B] \\ C \end{array} \quad (A \text{ does not cut})}{C}$$

264 As anticipated in the introduction these lemmas contradict the commutativity of disjunction.

265 4 Stack-based semantics

266 The stack-based semantics we present is very close to what is implemented by the ELPI interpreter.
 267 This semantics is similar to the one proposed by Pusch in [21], which is in turn closely related to
 268 the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in
 269 Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract
 270 Machine [27, 1].

271 The execution state is a stack of alternatives. Each alternative is a list of goals. Intuitively it
 272 amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a
 273 substitution with all the goals to be solved.

274 Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually
 275 recursive, but these alternatives have a very different role. They have to be understood as pointers
 276 to lower levels of the stack itself. This datum is used to implement the cut, i.e. to prune the stack
 277 of alternatives.

Definition `save_gs a g b :=`
`[seq (x, a) | x <- b] ++ g.`

Definition `save_as a g b :=`
`[seq (x.1, save_gs a g x.2) | x <- b].`

Definition `stepS p v t σ a g :=`
`let (v', r) := backchain p v t σ in`
`(v', save_as a g r).`

$$\begin{array}{c}
 \frac{}{\text{runS } p \ v \ ((\sigma, \emptyset) :: a) \cdot (\sigma, a)} \text{StopS} \qquad \frac{\text{runS } p \ v \ ((\sigma, g) :: ca) \ r}{\text{runS } p \ v \ ((\sigma, (\text{cut}, ca) :: g) :: _) \ r} \text{CutS} \\
 \\
 \frac{\text{stepS } p \ v \ t \ \sigma \ a \ g = (v_1, a') \quad \text{runS } p \ v_1 \ (a' ++ a) \ r}{\text{runS } p \ v \ ((\sigma, (\text{call } t, _) :: g) :: a) \ r} \text{CallS} \quad \frac{}{\text{runS } p \ _ \ \emptyset \ \square} \text{FailS}
 \end{array}$$

■ **Figure 8** Rule system for runS

Inductive `alts := nilA | consA of (Σ * goals) & alts`
with `goals := nilG | consG of (Atom * alts) & goals .`

278 The stack-based semantics is described by the `runS` inductive predicates.

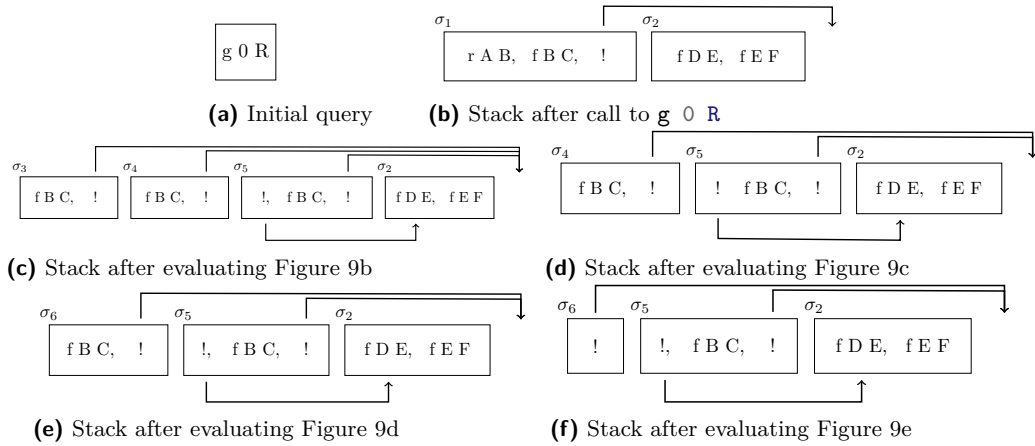
279 **Inductive** `runS (p: ℙ) : ℱv → alts → option (Σ * alts) → Prop` (1)

280 The `runS` predicates relates a set of variables and a list of alternatives to optional result. $\square$
 281 stands for failure, i.e., no solution, whereas $\cdot(\sigma, a)$ represent success with σ begin the solution and a
 282 as the list of non-yet-explored alternatives.

283 The rule system for `runS` is given in Figure 8 and is made by 4 cases. We distinguish two main
 284 cases. If we run out of alternatives we fail: (*FailS*) stops the execution and return $\square$. If the list of
 285 alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is
 286 empty, (*StopS*) stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h
 287 is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t
 288 is a cut rule (*CutS*) continues to evaluate the tail of h under with alternatives ca . Finally, when
 289 t is a call, rule (*CallS*) backchains: for each rule it pairs each premise with the current stack of
 290 alternatives, and prepends the obtained goals to the tail of h . Note that if backchain returns the empty
 291 list, (*CallS*) second premise amounts to exploring the next item in the current stack.

292 4.1 Example of execution

293 We propose an example of the execution of `runS` in Figure 9. The arrows in the images represent
 294 the cut-to pointers. We have not drawn arrows coming out of call atoms since the interpreter



■ **Figure 9** Example of execution

ignores the cut-alternatives in this case. The evolution of the stack in Figure 9 mirrors the example in Section 2.1: we evaluate the first goal of the first alternative. During backchaining, we add a pointer to the remaining alternatives to each new cut operator. If a cut is evaluated, we replace the remaining alternatives with the cut-to alternatives. For example, in Figure 9f, evaluating the cut discards the second and third elements in the stack. Backtracking is performed by simply consuming the first alternative from the stack, as shown in Figure 9d.

4.2 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in Section 3.4 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \dots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \dots, a_n$ to the alternatives $b_1, \dots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \dots, b_m$ is a suffix of $a_1, \dots, a_n$. If $a_1, \dots, a_n$ were generated before the call whose execution generates the cut in a_0 , then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

5 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called `t2l`, but we do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in Section 5.2) we need to define the notion of *valid tree*. The `tree` datatype indeed contains trees that cannot be generated by `runT` via `step` or `prune`, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees

The class of valid trees is captured by the `valid_tree` function shown in Figure 10. While all base cases of `tree` are valid, we need to analyze the `Or` and `And` constructors more carefully.

For the `Or` constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the `K0` tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the `base_or` predicate to check that `B` is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a `call`.

For the `And` constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. `B` must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the `big_and` procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by `backchain` into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

5.2 From trees to stacks

The `t2l` recursive function translates a tree to a stack. For space constraints Figure 10 only gives the base cases, while we describe the recursive cases in the following text.

The function `t2l` takes as input the tree t_0 , a substitution σ_0 , and a list of choice points cp . These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an `Or` node. The substitution is used to determine under which substitution the subtree being computed lives.

```

Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO => T
  | Or □ _ B => valid_tree B
  | Or ·A _ B => valid_tree A &&
    ((B == KO) || B.base_or B)
  | And A B0 B => valid_tree A &&
    if success A then valid_tree B
    else B == big_and B0
  end.

Fixpoint t2l t σ (cp: alts) : alts :=
  match t with
  | OK      => [(σ, ∅)]
  | KO      => ∅
  | TA a    => [(σ, [(a, ∅)])]
  ...

```

■ **Figure 10** Valid tree and Tree to list

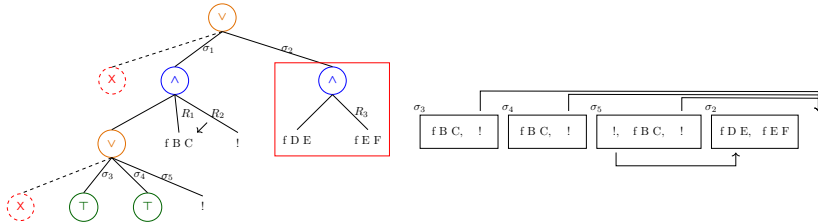
342 The OK node represents one successful alternative; therefore it is translated into a singleton
 343 list whose only element has an empty list of goals. The KO node represents failure, hence it is
 344 mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing
 345 one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to
 346 list cannot point to cp : atoms are not right-uncle subtrees, whereas cp represents alternatives that
 347 will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore
 348 determined later, once the trees corresponding to sibling subtrees have been inspected.

349 The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B .
 350 The alternatives of B are valid choice points for A and should be passed to the translation of A .
 351 The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level
 352 below cp (i.e. in A or B) should add cp as its cut-to alternatives.

353 The translation of $A \wedge_{B_0} B$ starts with the translation of A . If the translation of A is an empty list
 354 we return the empty list of alternatives without even touching B nor B_0 , since an empty translation
 355 for A indicates failure and this in turn implies the failure of the entire conjunction. When the
 356 translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g ,
 357 whereas B_0 is the continuation for each alternative in a .

358 5.2.1 Example of $t2l$

359 The translation of the trees in Figure 6 is given in Figure 9. The letters in the figures relate the
 360 trees to the corresponding letters in the stack figures. For example, the tree 6b is translated into
 361 9b. We note that the two trees 6d.1 and 6d.2 are both translated into 9d, showing that $t2l$ is not
 362 injective; that is, there are different trees that map to the same stack. This means that there are
 363 transitions in runT that are not visible in runS .



364

365 As an example of a call to $t2l$, we take the tree and the stack above, which are respectively taken
 366 from Figures 6c and 9c. The red arrows points the head of an alternative in the tree to the head of
 367 the corresponding cell in the stack. We focus on the deepest Or node in the tree. This subtree
 368 is a disjunction with four children. The first is ignored, since it is $\square$. The success node below σ_3
 369 has $f \ B \ C$ and the cut operator as continuation, corresponding to the first cell in the stack. We

note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals $\mathbf{f} \ B \ C, !$; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2 , since it is one **Or**-layer above its right uncles.

5.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

► **Definition 12** ($\text{runT}' p \ v \ \sigma \ t \ r$). $\exists b \ v', \ \text{runT} \ p \ v \ \sigma \ t \ r \ b \ v'$.

From Figures 6 and 9, we observe that, upon backtracking, **runT** may perform more steps than **runS** before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

► **Lemma 13** (pruneF_t2l). $\forall t \ t' \ b, \ \text{valid_tree} \ t \rightarrow \text{failed} \ t \rightarrow \text{prune} \ b \ t = \cdot t' \rightarrow \forall l \ \sigma, \ t2l \ t \ \sigma \ l = t2l \ t' \ \sigma \ l$.

Proof. By induction on the tree t . ◀

dire di v

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

► **Theorem 14** (tree_to_elpi). $\forall p \ t \ \sigma \ r, \ \text{let } v := \text{vars_tree } t \ \backslash \backslash \text{vars_sigma } \sigma \text{ in}$

$\text{valid_tree} \ t \rightarrow$
 $\text{runT}' \ p \ v \ \sigma \ t \ r \rightarrow$
 $\text{let } a := t2l \ t \ \sigma \ \emptyset \text{ in}$
 $\text{let } r' := \text{if } r \text{ is } \cdot(\sigma', t) \text{ then } \cdot(\sigma', t2l \ (\text{odflt } K0 \ t) \ \sigma \ \emptyset)$
 $\text{else } \square \text{ in}$
 $\text{runS} \ p \ v \ a \ r'$.

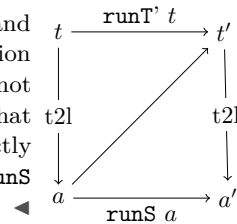
Proof. We proceed by induction on the execution of **runT'**. There are four cases to consider. The first case arises from *StopT*; it deals with successful trees. A successful tree must start with an empty list, and therefore we conclude using *StopS*. The case for *StepT* requires treating two subcases: **step** evaluates either a cut or a call. Accordingly, we apply *CutS* or *CallS*, followed by the induction hypothesis. The case for *BackT* is silent: it represents the non-injective behavior of **t2l**; however, thanks to Lemma 13 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation *FailT*. It is easily proved using *FailS* and the fact that if **prune** returns $\square$ when trying to erase failure in t , then **t2l** applied to t returns the empty list. ◀

The converse direction is stated following [2]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns a unexplored tree that translates to the unexplored stack.

► **Theorem 15** (elpi_to_tree). $\forall p \ v \ a \ r, \ \text{runS} \ p \ v \ a \ r \rightarrow$

$\forall \sigma \ t, \ \text{valid_tree} \ t \rightarrow t2l \ t \ \sigma \ \emptyset = a \rightarrow$
 $\text{if } r \text{ is } \cdot(\sigma', a') \text{ then}$
 $\exists t', \ \text{runT}' \ p \ v \ \sigma \ t \cdot(\sigma', t') \wedge t2l \ (\text{odflt } K0 \ t') \ \sigma \ \emptyset = a'$
 $\text{else } \text{runT}' \ p \ v \ \sigma \ t \ \square$.

Proof. The proof is based on the idea explained in [2, Section 3.5.1] and depicted in the figure on the right: we have a state t whose translation is a , we proceed by induction on the execution of **runS**. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to the stack state is a' . This proof strategy is indirectly providing a kind of **t2l** function by combining the execution of the **runS** and **t2l**.



400 Stating the converse direction in a symmetric way would require a function `l2t` transforming a
 401 stack a into a tree t . Writing this function is far from trivial, since the structure of the tree is lost by
 402 `save_as` and `runT` that intuitively keep the state as a big disjunction of conjunctions by distributing
 403 conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is
 404 equally intricate.

405 The constructive nature of the logic of Rocq tells us that the proof above provides an effective
 406 function translating stacks into trees. However, that function also uses an execution trace of the
 407 stack-based semantics, from which it recovers the missing tree structure.

408 6 Case study: determinacy analysis

409 An interesting application of the tree semantics is determinacy checking, introduced in version 3 of
 410 the ELPI language [12]. The checker takes as input the signatures of the predicates declared by the
 411 user and verifies that the rules of a deterministic predicate generate at most one solution; we call
 412 this particular kind of predicate a function. Thanks to this static verification, the user can better
 413 control unexpected backtracking statically.

414 A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees
 415 that at most one rule can be successfully applied to any given query, whereas *rule checking* ensures
 416 that each rule does not create choice points, for example by calling non-deterministic predicates.

417 Determinacy checking is strongly related to the modes of predicates: we differentiate between
 418 input and output arguments. A deterministic call relates to its inputs only one output. In the
 419 literature, we find several works about determinacy checking, namely [22, 17, 7]. These works need a
 420 mode checker statically ensuring that in a query with ground inputs, each recursive call is performed
 421 with ground inputs.

422 In our mechanization, and in particular in the ELPI language, we adopt a special unification
 423 routine for input argument, called `matching` [1, 28]. Similarly to `unify`, `matching` takes the query-
 424 term t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable
 425 in the query.

426 The function `matching` has the following property:

427 ► **Axiom 1** (`matching_disj`). $\forall \sigma \sigma' t_1 t_2, \text{vars } t_1 \# \text{domf } \sigma \rightarrow \text{vars } t_1 \# \text{vars } t_2 \rightarrow$
 $\text{matching } t_1 t_2 \sigma = \sigma' \rightarrow \exists e, \text{domf } \sigma' = \text{domf } \sigma \setminus e \wedge e \subseteq \text{vars } t_2.$

428 If the two terms are disjoint and the variables in the support of σ are disjoint from the variables
 429 in t_1 , then, if matching succeeds, only the variables in t_2 are assigned; that is, $t_1 = \sigma' t_2$, and the
 430 variables in t_1 are not assigned in σ' .

431 The `backchain` function uses `matching` or `unify` on the arguments q depending if the argument
 432 is in *input* or *output* mode. We assume two important properties of our unification routines.

433 ► **Axiom 2** (`unif_trans`). $\forall t_1 t_2 t_3 \sigma, \text{unify } t_1 t_2 \sigma \rightarrow \text{unify } t_2 t_3 \sigma \rightarrow \text{unify } t_1 t_3 \sigma.$

434 ► **Axiom 3** (`match_unif`). $\forall t_1 t_2 \sigma, \text{matching } t_1 t_2 \sigma \rightarrow \text{unify } t_1 t_2 \sigma.$

435 We also assume that refreshing variables does not affect unification. In particular a refreshing
 436 renaming for a term t is a function from variables to variables that: (i) is defined on all the variables
 437 in t ; (ii) is injective; (iii) has non overlapping domain and codomain. .

438 ► **Definition 16** (`refresh_for x t`). $(\text{vars } t \subseteq \text{domf } x) \wedge \text{injective } x.$

439 ► **Axiom 4** (`unif_ren`). $\forall x y z w t_1 t_2,$
 $\text{refresh_for } w t_1 \rightarrow \text{refresh_for } y t_2 \rightarrow \text{refresh_for } z t_1 \rightarrow \text{refresh_for } x t_2 \rightarrow$
 $\text{codomf } w \# \text{vars } t_1 \mid \text{vars } (\text{rename } y t_2) \rightarrow$
 $\text{codomf } z \# \text{vars } t_1 \mid \text{vars } (\text{rename } x t_2) \rightarrow$
 $\text{unify } (\text{rename } w t_1) (\text{rename } y t_2) \varepsilon \rightarrow \text{unify } (\text{rename } z t_1) (\text{rename } x t_2) \varepsilon.$

MANCA
 QUESTA
 PARTE, dire
 cosa sia SHARP
 e ren

In Figure 1, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that **f** and **g** are expected to be functions, as specified by the **func** keyword, whereas **r** is a relation, as indicated by the **pred** keyword. The **->** symbol separates inputs from outputs; therefore, all three predicates are expected to take an **int** as input and return an **int** as output.

The signatures of the program are stored in the **sig** field of the program and are encoded as described in [12]; that is, similarly to the code in Figure 1, we use arrows for term application, data types to encode base types such as *int*, and two dedicated symbols to distinguish deterministic from non-deterministic predicates.

A program execution behaves deterministically if, given a program, a substitution, and a tree to the **runT** inductive, the execution either fails or, if it succeeds, the tree of alternatives is $\square$.

► **Definition 17** ($\text{is_det } p \sigma v t$). $\forall r, \text{runT}' p v \sigma t r \rightarrow r = \square \vee \exists \sigma, r = \cdot(\sigma, \square)$.

The theorem we want to prove is the following one:

► **Theorem 18** (det_check_call). $\forall p \sigma t v,$
 $\text{check_}\mathbb{P} p \rightarrow \text{tm_is_det } p.(sig) t \rightarrow \text{is_det } p \sigma v (\text{Todo } (call t)).$

Here, $\text{check_}\mathbb{P}$ denotes the function that checks the program with respect to mutual exclusion and rule checking, and tm_is_det denotes the function that tests whether the head of the term refers to a rigid constant with a deterministic type.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about **Todo** trees (i.e. atoms), whereas we need it on any tree.

To address this issue, we introduce the notion of “deterministic trees” (**det_tree**), show that a tree satisfying this condition enjoys the desired properties, and then prove the following more general theorem:

► **Lemma 19** (det_check_tree). $\forall \sigma v p t, \text{check_}\mathbb{P} p \rightarrow \text{det_tree } p.(sig) t \rightarrow \text{is_det } p \sigma v t.$

Our target theorem **det_check_call** is a corollary of **det_check_tree**.

7 Related work

We studied a PROLOG semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the ELPI interpreter, which is in turn inspired by B-Prolog’s “matching-clauses” [1, 28]. The two semantics we presented (and proved equivalent) also apply to standard PROLOG if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the development only as follows: Axiom 1 does not require the query q to be ground. Adapting our results to standard PROLOG therefore requires the insertion of two additional hypotheses in Theorem 18. First, one has to require that t has ground inputs. Second, that the program p is well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in Section 6 still apply.

The only mechanization of Prolog’s semantics we could find is the one by Pusch in ISABELLE/HOL [21] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work starts from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [27, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in Section 4 that pushed us to develop a more explicit semantics (with respect to cut). In a sense, our work is complementary, showing the stack-based semantics equivalent to a more optimized one.

Another relevant work is by King [16], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features our paper covers, but it would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [2], and we thank Y. Bertot for insightful discussions on the topic. In [2], Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of Theorem 15.

8 Conclusion

This paper describes two operational semantics for Prolog with cut. The first semantics is based on And-Or trees: it is well suited to graphically illustrating the scope of cut, and to proving lemmas about the impact of its evaluation when reasoning about disjunctions. The second semantics is based on a stack of alternatives: it is computationally more efficient, but it loses the structural expressiveness of the tree-based semantics. Thanks to a dedicated function, we translate trees into stacks and prove the two semantics equivalent. This stack-based semantics corresponds to the first-order fragment of ELPI.

Using the tree-based semantics, we mechanize the first-order part of the determinacy checker that has been available in ELPI since version 3.0, and we prove that a call to a deterministic predicate returns at most one solution.

The code development consists of approximately 2,500 lines for the specifications and about 5,000 lines of *ssreflect* proofs. About 30% of the code is dedicated to the tree semantics, such as the proofs in Section 3.4. About 15% of the code is devoted to the stack semantics, and 35% to the equivalence proof. The remaining 20% of the code is for the determinacy checker. The development took approximately 8 months.

To fully model the ELPI language, we would need to account for higher-order variables, i.e. variables representing predicates. This extension is ongoing and largely orthogonal to the present paper, since cut and higher-order features do not interact in subtle ways.

References

- 1 Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- 4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
- 5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPICs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.FSCD.2020.34>, doi:10.4230/LIPICs.FSCD.2020.34.
- 6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:Debray1988.
- 7 Saumya K. Debray and David S. Warren. Functional computations in logic programs. volume 11, page 451–481. Association for Computing Machinery, July 1989. doi:10.1145/65979.65984.

- 537 **8** Gilles Dowek. *Higher-order unification and matching*, page 1009–1062. Elsevier Science
538 Publishers B. V., NLD, 2001.
- 539 **9** Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast,
540 embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages
541 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/
542 978-3-662-48899-7_32.
- 543 **10** Davide Fisore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq*
544 *Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 545 **11** Davide Fisore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-
546 language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024.
547 doi:10.1145/3678232.3678233.
- 548 **12** Davide Fisore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic
549 programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects*
550 *of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- 551 **13** Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality
552 tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing
553 Machinery, 2023. doi:10.1145/3573105.3575683.
- 554 **14** Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory
555 in higher order constraint logic programming. In *Mathematical Structures in Computer*
556 *Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/
557 S0960129518000427.
- 558 **15** Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least
559 Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors,
560 *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leib-*
561 *niz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany,
562 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: [https://drops.dagstuhl.](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27)
563 [de/entities/document/10.4230/LIPIcs.ITP.2025.27](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27), doi:10.4230/LIPIcs.ITP.2025.27.
- 564 **16** Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. volume 11, pages
565 537–553. Cambridge University Press, 2011.
- 566 **17** Lunjin Lu and Andy King. Determinacy inference for logic programs. volume 3444, pages
567 108–123, 04 2005. doi:10.1007/978-3-540-31987-0_9.
- 568 **18** math-comp. finmap — finite maps for mathcomp, 2026. URL: [https://github.com/](https://github.com/math-comp/finmap)
569 [math-comp/finmap](https://github.com/math-comp/finmap).
- 570 **19** Dale Miller. A logic programming language with lambda-abstraction, function variables, and
571 simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 572 **20** Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge
573 University Press, 2012.
- 574 **21** Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris
575 Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors,
576 *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer
577 Berlin Heidelberg.
- 578 **22** Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm
579 of mercury, an efficient purely declarative logic programming language. volume 29,
580 pages 17–64, 1996. High-Performance Implementations of Logic Programming Systems.
581 URL: <https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684>,
582 doi:10.1016/S0743-1066(96)00068-4.
- 583 **23** SWI-Prolog Documentation. Predicate !/0 — cut (built-in predicate), 2026. Accessed via SWI-
584 Prolog PLDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.

- 585 **24** Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog
586 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
587 2018. URL: <https://inria.hal.science/hal-01637063>.
- 588 **25** Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
589 *LIPICs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
590 doi:10.4230/LIPICs.CVIT.2016.23.
- 591 **26** Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
592 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 593 **27** David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
594 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
595 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/](https://www.sri.com/wp-content/uploads/2021/12/641.pdf)
596 [2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).
- 597 **28** Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*,
598 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.